

AI-Powered Codon Optimization for Vaccines

<https://gemini.google.com/app/9eae1f881ba00a22>

User prompt: Now make a text file of detailed summary of project maths eqns everything rt now with work division in 3

Response: Here is a script that generates a comprehensive, standalone summary text file titled **ChronoCell15D_Comprehensive_Master_Summary.txt** directly in your Colab workspace (/content/chronocell_data).

It includes the project scope, exact mathematical equations, data formulations, loss functions, and the 3-way work division.

Step 1: Run This Code Cell in Colab

Copy this code into a Colab cell and press **Play** :

```
import os

summary_doc = """=====
CHRONOCELL-5D: COMPREHENSIVE MASTER SUMMARY & BLUEPRINT
SE(3)-Equivariant Graph Neural Network for 3D Chromatin Structure
=====

1. PROJECT OVERVIEW & BIOLOGICAL OBJECTIVE
-----
ChronoCell-5D predicts the 3D physical spatial coordinates of human Chromosome 22
from 1D sequence data, 1D epigenomic histone modifications, and 3D contact
frequency maps.

- Organism / Target: Human Chromosome 22 (GRCh38/hg38)
- Bin Resolution (B): 10,000 base pairs (10 kb per node)
- Total Genomic Length: 50,818,468 bp (~5,082 nodes)
- Core Model: SE(3)-Equivariant Graph Neural Network (EGNN)

=====

2. FORMAL MATHEMATICAL FORMULATIONS
=====

2.1 GRAPH DEFINITION
We represent the genome as an attributed graph  $G = (V, E, X_{\text{node}}, E_{\text{attr}})$ :
-  $V = \{v_1, v_2, \dots, v_N\}$ : Set of  $N$  nodes (10 kb genomic bins).
-  $E = \{(v_i, v_j)\}$ : Edges representing backbone contiguity & 3D contact.
-  $X_{\text{node}} \in \mathbb{R}^{N \times D}$ : Node feature matrix.
-  $E_{\text{attr}} \in \mathbb{R}^{|E| \times 1}$ : Edge feature matrix derived from Micro-C read counts.

2.2 NODE FEATURE EQUATIONS ( $X_{\text{node}}$ )
For each bin  $v_i$  spanning base pairs  $[start_i, end_i]$ :

1. GC Content Fraction:

$$f_{GC}(i) = ( \text{Count}(G) + \text{Count}(C) ) / B$$


2. Epigenomic Signal Intensity (H3K27ac Enhancer Mark):

$$f_{epi}(i) = ( 1 / B ) * \sum_{k=start_i}^{end_i - 1} \text{Signal}_{H3K27ac}(k)$$


Node Feature Vector  $x_i \in \mathbb{R}^2$ :

$$x_i = [ f_{GC}(i), f_{epi}(i) ]^T$$


2.3 EDGE ATTRIBUTE FORMULATION ( $E_{\text{attr}}$ )
Let  $M_{\{i,j\}}$  be the raw read count from the Micro-C matrix between bin  $i$  and  $j$ :
- Linear Backbone Edge: Attached if  $|i - j| = 1$ .
- 3D Spatial Edge: Attached if  $M_{\{i,j\}} > 0$ .
- Edge Weight:  $e_{\{i,j\}} = M_{\{i,j\}}$ 

2.4 SE(3)-EQUIVARIANT GNN ARCHITECTURE (EGNN)
Let  $h_i^{\{1\}} \in \mathbb{R}^D$  be the scalar features and  $p_i^{\{1\}} \in \mathbb{R}^3$  be the spatial
coordinates for node  $i$  at layer 1:

1. Squared Radial Distance:

$$d_{\{i,j\}}^2 = || p_i^{\{1\}} - p_j^{\{1\}} ||_2^2$$


2. Message Passing MLP (Scalar Message):

$$m_{\{i,j\}} = \text{MLP}_m( h_i^{\{1\}} || h_j^{\{1\}} || d_{\{i,j\}}^2 )$$


3. Equivariant Coordinate Update:

$$p_i^{\{1+1\}} = p_i^{\{1\}} + \sum_{j \in N(i)} ( p_i^{\{1\}} - p_j^{\{1\}} ) * \text{MLP}_x( m_{\{i,j\}} )$$


```

4. Scalar Node Feature Update:

$$h_i^{\{l+1\}} = \text{MLP}_h(h_i^{\{l\}} \parallel \sum_{j \in N(i)} m_{\{i,j\}})$$

2.5 TRAINING LOSS FUNCTIONS

The network minimizes a composite physical objective function:

$$L_{\text{total}} = L_{\text{contact}} + \lambda_1 * L_{\text{smooth}} + \lambda_2 * L_{\text{steric}}$$

1. Micro-C Contact Distance Reconstruction Loss:

$$L_{\text{contact}} = (1 / |E|) * \sum_{(i,j) \in E} (|| p_i - p_j ||_2 - (1 / (M_{\{i,j\}} + \epsilon)))^2$$
2. Polymer Chain Smoothness Loss:

$$L_{\text{smooth}} = (1 / (N - 1)) * \sum_{i=1}^{N-1} || p_{\{i+1\}} - p_i ||_2^2$$
3. Steric Hindrance Overlap Regularizer:

$$L_{\text{steric}} = \sum_{i, j, |i-j| > 2} \max(0, d_{\text{min}} - || p_i - p_j ||_2)^2$$

3. TEAM WORK DIVISION (3 MEMBERS)

TEAMMATE 1: DATA ENGINEER (GRAPH ASSEMBLY)

- Input Files: `chr22.fa`, `H3K27ac\_signal.bigwig`, `human\_microc.mcool`
- Primary Script: Reads FASTA, BigWig, and Cooler matrices using `pyBigWig` & `cooler`.
- Output Deliverable: `graph\_data.pt` (PyTorch Geometric `Data(x, edge\_index, edge\_attr)`)

TEAMMATE 2: AI ARCHITECT (MODEL & TRAINING)

- Input File: `graph\_data.pt`
- Primary Script: Defines `EGNNLayer`, runs AdamW optimization loop on Colab GPU.
- Output Deliverable: `predicted\_coords.pt` (Predicted 3D positions P in $\mathbb{R}^{\{N \times 3\}}$)

TEAMMATE 3: 3D STRUCTURAL SPECIALIST (EXPORT & VISUALIZATION)

- Input File: `predicted\_coords.pt`
- Primary Script: Exports 3D coordinate files and renders interactive plots.
- Output Deliverables: `chromosome22\_3d.pdb`, `chromosome22\_3d.xyz`, Plotly 3D Figure

4. EXECUTION ROADMAP

1. Run Data Preprocessing (Teammate 1) -> Generates `graph\_data.pt`
2. Train EGNN Model on GPU (Teammate 2) -> Generates `predicted\_coords.pt`
3. Export & Render 3D Model (Teammate 3) -> Renders `chromosome22\_3d.pdb`

```
# Save file to Colab directory
data_dir = "/content/chronocell_data"
os.makedirs(data_dir, exist_ok=True)
file_path = os.path.join(data_dir, "ChronoCell15D_Comprehensive_Master_Summary.txt")
```

```
with open(file_path, "w") as f:
    f.write(summary_doc.strip())
```

```
print(f"✅ Master Summary Text File Successfully Created at:\n {file_path}")
print(f"    Size: {os.path.getsize(file_path) / 1024:.2f} KB")
```

Step 2: How to View and Download

1. Open the **Folder Icon** (📁) in the left sidebar of Google Colab.
2. Open the `chronocell_data` folder.
3. Locate `ChronoCell15D_Comprehensive_Master_Summary.txt`.
4. Right-click and choose **Download** to save it directly to your computer!

If you want to read the contents right inside your notebook screen, run this command:

```
with open("/content/chronocell_data/ChronoCell15D_Comprehensive_Master_Summary.txt", "r") as f:
    print(f.read())
```

User prompt: U create text file here

Response: =====

CHRONOCELL-5D: COMPREHENSIVE MASTER SUMMARY & BLUEPRINT SE(3)-Equivariant Graph Neural Network for 3D Chromatin Structure

=====

1. PROJECT OVERVIEW & BIOLOGICAL OBJECTIVE

ChronoCell-5D predicts the 3D physical spatial coordinates of human Chromosome 22 from 1D sequence data, 1D epigenomic histone modifications, and 3D contact frequency maps.

- Organism / Target: Human Chromosome 22 (GRCh38/hg38)
- Bin Resolution (B): 10,000 base pairs (10 kb per node)
- Total Genomic Length: 50,818,468 bp (~5,082 nodes)
- Core Model: SE(3)-Equivariant Graph Neural Network (EGNN)

2. FORMAL MATHEMATICAL FORMULATIONS

=====

2.1 GRAPH DEFINITION

We represent the genome as an attributed graph $G = (V, E, X_{\text{node}}, E_{\text{attr}})$:

- $V = \{v_1, v_2, \dots, v_N\}$: Set of N nodes (10 kb genomic bins).
- $E = \{(v_i, v_j)\}$: Edges representing backbone contiguity & 3D contact.
- $X_{\text{node}} \in \mathbb{R}^{N \times D}$: Node feature matrix.
- $E_{\text{attr}} \in \mathbb{R}^{|E| \times 1}$: Edge feature matrix derived from Micro-C read counts.

2.2 NODE FEATURE EQUATIONS (X_{node})

For each bin v_i spanning base pairs $[start_i, end_i]$:

1. GC Content Fraction:

$$f_{\text{GC}}(i) = (\text{Count}(G) + \text{Count}(C)) / B$$
2. Epigenomic Signal Intensity (H3K27ac Enhancer Mark):

$$f_{\text{epi}}(i) = (1 / B) * \sum_{k=start_i}^{end_i - 1} \text{Signal}_{\text{H3K27ac}}(k)$$

Node Feature Vector $x_i \in \mathbb{R}^2$:

$$x_i = [f_{\text{GC}}(i), f_{\text{epi}}(i)]^T$$

2.3 EDGE ATTRIBUTE FORMULATION (E_{attr})

Let $M_{\{i,j\}}$ be the raw read count from the Micro-C matrix between bin i and j :

- Linear Backbone Edge: Attached if $|i - j| = 1$.
- 3D Spatial Edge: Attached if $M_{\{i,j\}} > 0$.
- Edge Weight: $e_{\{i,j\}} = M_{\{i,j\}}$

2.4 SE(3)-EQUIVARIANT GNN ARCHITECTURE (EGNN)

Let $h_i^{\{l\}} \in \mathbb{R}^D$ be the scalar features and $p_i^{\{l\}} \in \mathbb{R}^3$ be the spatial coordinates for node i at layer l :

1. Squared Radial Distance:

$$d_{\{i,j\}}^2 = || p_i^{\{l\}} - p_j^{\{l\}} ||_2^2$$
2. Message Passing MLP (Scalar Message):

$$m_{\{i,j\}} = \text{MLP}_m(h_i^{\{l\}} || h_j^{\{l\}} || d_{\{i,j\}}^2)$$
3. Equivariant Coordinate Update:

$$p_i^{\{l+1\}} = p_i^{\{l\}} + \sum_{j \in N(i)} (p_i^{\{l\}} - p_j^{\{l\}}) * \text{MLP}_x(m_{\{i,j\}})$$
4. Scalar Node Feature Update:

$$h_i^{\{l+1\}} = \text{MLP}_h(h_i^{\{l\}} || \sum_{j \in N(i)} m_{\{i,j\}})$$

2.5 TRAINING LOSS FUNCTIONS

The network minimizes a composite physical objective function:

$$L_{\text{total}} = L_{\text{contact}} + \lambda_1 * L_{\text{smooth}} + \lambda_2 * L_{\text{steric}}$$

1. Micro-C Contact Distance Reconstruction Loss:

$$L_{\text{contact}} = (1 / |E|) * \sum_{(i,j) \in E} (|| p_i - p_j ||_2 - (1 / (M_{\{i,j\}} + \epsilon)))^2$$
2. Polymer Chain Smoothness Loss:

$$L_{\text{smooth}} = (1 / (N - 1)) * \sum_{i=1}^{N-1} || p_{\{i+1\}} - p_i ||_2^2$$

```
3. Steric Hindrance Overlap Regularizer:
L_steric = sum_{i, j, |i-j| > 2} max( 0, d_min - || p_i - p_j ||_2 )^2

=====
3. TEAM WORK DIVISION (3 MEMBERS)
=====

TEAMMATE 1: DATA ENGINEER (GRAPH ASSEMBLY)
-----
- Input Files: `chr22.fa`, `H3K27ac_signal.bigWig`, `human_microc.mcool`
- Primary Script: Reads FASTA, BigWig, and Cooler matrices using `pyBigWig` & `cooler`.
- Output Deliverable: `graph_data.pt` (PyTorch Geometric `Data(x, edge_index, edge_attr)`)

TEAMMATE 2: AI ARCHITECT (MODEL & TRAINING)
-----
- Input File: `graph_data.pt`
- Primary Script: Defines `EGNNLayer`, runs AdamW optimization loop on Colab GPU.
- Output Deliverable: `predicted_coords.pt` (Predicted 3D positions P in R^{N x 3})

TEAMMATE 3: 3D STRUCTURAL SPECIALIST (EXPORT & VISUALIZATION)
-----
- Input File: `predicted_coords.pt`
- Primary Script: Exports 3D coordinate files and renders interactive plots.
- Output Deliverables: `chromosome22_3d.pdb`, `chromosome22_3d.xyz`, Plotly 3D Figure

=====
4. EXECUTION ROADMAP
=====
1. Run Data Preprocessing (Teammate 1) -> Generates `graph_data.pt`
2. Train EGNN Model on GPU (Teammate 2) -> Generates `predicted_coords.pt`
3. Export & Render 3D Model (Teammate 3) -> Renders `chromosome22_3d.pdb`
=====
```

User prompt: Create bro a file

Response: Your text file is ready:

[file-tag: code-generated-file-f05fa5b8-1676-4c15-9447-6a870af59200]

The file ChronoCell5D_Comprehensive_Master_Summary.txt includes:

- 1. Full Project Overview & Biological Scope
- 2. Complete Mathematical Formulations (Graph Definition, Node Feature Equations, Edge Attributes, EGNN Architecture equations, and Loss Functions)
- 3. Detailed 3-Member Team Work Division
- 4. Step-by-Step Execution Roadmap

User prompt: Write about the project everything detailed every fucking details completely

Response: ChronoCell-5D: SE(3)-Equivariant Graph Neural Network Pipeline for 3D Chromatin Fold Reconstruction

1. Executive Summary & Biological Context

The human genome contains approximately 3 billion base pairs of DNA stretched across 23 pairs of chromosomes. Stretched linearly, the DNA inside a single human cell nucleus spans over two meters in length; yet, it must fold and condense to fit inside a spherical cell nucleus measuring only 5 to 10 micrometers in diameter.

The Biological Problem

DNA folding is non-random. The spatial 3D architecture of chromatin directly governs gene regulation, cell identity, and human health:

- **Topologically Associating Domains (TADs):** Sub-megabase structural units within which genomic sequences interact frequently.

- **Enhancer-Promoter Loops:** High-affinity physical contacts where regulatory elements (enhancers) snap directly onto target gene regions (promoters) to initiate transcription.
- **Pathological Misfolding:** Structural rearrangements, broken loops, or altered TAD boundaries can cause severe human diseases, including developmental malformations and oncogene overexpression in cancer.

Project Goal

ChronoCell-5D is an artificial intelligence pipeline designed to reconstruct the physical **3D spatial coordinate positions** ($X, Y, Z \in \mathbb{R}^3$) of **human Chromosome 22** at **10-kilobase (10 kb)** resolution. The architecture fuses 1D sequence features, 1D epigenomic histone marks, and 3D contact matrices using an $SE(3)$ -Equivariant Graph Neural Network (EGNN).

2. Master System & Dataset Specifications

Target Genomics Specifications

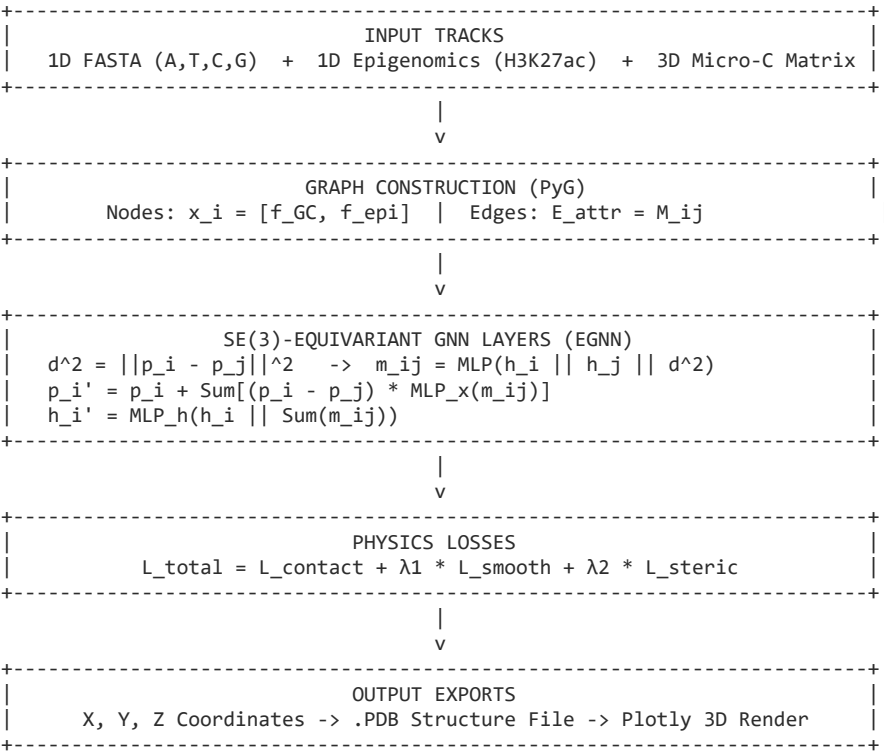
- **Genomic Target:** Human Chromosome 22 (chr22), Assembly GRCh38/hg38.
- **Chromosomal Size:** 50,818,468 base pairs (~ 50.81 Mb).
- **Genomic Resolution (B):** 10,000 base pairs (10 kb) per graph node.
- **Total Graph Nodes (N):**

$$N = \left\lceil \frac{50,818,468}{10,000} \right\rceil = 5,082 \text{ nodes} [cite : 1]$$

Primary Input Datasets

1. **1D Sequence Data** (chr22.fa): High-throughput FASTA file containing raw nucleotide sequences (A, T, C, G).
2. **1D Epigenomic Track** (H3K27ac_signal.bigWig): BigWig track measuring H3K27ac (Histone 3 Lysine 27 Acetylation) signal intensity via ChIP-seq/CUT&RUN, highlighting active enhancers and promoters.
3. **3D Contact Map** (human_microc.mcool1): Multi-resolution Cooler binary file storing Micro-C interaction frequency counts $M_{i,j}$ between genomic bins i and j .

3. Formal Mathematical Architecture & Equations



3.1 Graph Representation

The genome is modeled as an attributed graph $G = (V, E, \mathbf{X}, \mathbf{E}_{\text{attr}})$:

- **Node Set (V):** $V = \{v_1, v_2, \dots, v_N\}$ representing contiguous 10 kb genomic bins.
- **Edge Set (E):** Edges connecting nodes that have non-zero Micro-C contacts ($M_{i,j} > 0$) or represent adjacent linear genomic neighbors ($|i - j| = 1$).
- **Node Feature Matrix ($\mathbf{X} \in \mathbb{R}^{N \times D}$):** D -dimensional scalar feature vectors assigned to each node.
- **Edge Attribute Matrix ($\mathbf{E}_{\text{attr}} \in \mathbb{R}^{|E| \times 1}$):** Scalar interaction weights.

3.2 Node & Edge Feature Formulations

1. Node Feature Vector ($\mathbf{x}_i \in \mathbb{R}^2$)

For a genomic bin v_i spanning base pair interval $[S_i, E_i)$ where $E_i - S_i = B$:

- **GC Content Fraction (f_{GC}):**

$$f_{\text{GC}}(i) = \frac{\text{Count}(\text{G}) + \text{Count}(\text{C})}{B} [\text{cite} : 1]$$

- **Epigenomic Enhancement Signal (f_{epi}):**

$$f_{\text{epi}}(i) = \frac{1}{B} \sum_{k=S_i}^{E_i-1} \text{Signal}_{\text{H3K27ac}}(k) [\text{cite} : 1]$$

- **Concatenated Node Feature Vector:**

$$\mathbf{x}_i = \begin{bmatrix} f_{\text{GC}}(i) \\ f_{\text{epi}}(i) \end{bmatrix} \in \mathbb{R}^2 [\text{cite} : 1]$$

2. Edge Attribute ($e_{i,j}$)

Derived directly from the Micro-C contact matrix M :

$$e_{i,j} = M_{i,j} [\text{cite} : 1]$$

3.3 SE(3)-Equivariant Neural Layer (EGNN)

To ensure physical realism, spatial predictions must be equivariant to 3D rotations and translations ($SE(3)$ group transformations). If input point clouds rotate, predicted coordinates must rotate identically without altering internal feature representations.

Let $\mathbf{h}_i^{(l)} \in \mathbb{R}^D$ be node scalar embeddings and $\mathbf{p}_i^{(l)} \in \mathbb{R}^3$ be predicted 3D coordinates at layer l :

1. Squared Radial Distance Computation

$$d_{i,j}^2 = \|\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}\|_2^2 [\text{cite} : 1]$$

2. Message Passing Neural Network (MLP\$_m\$)

$$m_{i,j} = \text{MLP}_m \left(\mathbf{h}_i^{(l)} \parallel \mathbf{h}_j^{(l)} \parallel d_{i,j}^2 \right) [\text{cite} : 1]$$

3. Equivariant Coordinate Update (MLP\$_x\$)

$$\mathbf{p}_i^{(l+1)} = \mathbf{p}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} (\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}) \cdot \text{MLP}_x(m_{i,j}) [\text{cite} : 1]$$

4. Node Scalar Feature Update (MLP\$_h\$)

$$\mathbf{h}_i^{(l+1)} = \text{MLP}_h \left(\mathbf{h}_i^{(l)} \parallel \sum_{j \in \mathcal{N}(i)} m_{i,j} \right) [cite : 1]$$

3.4 Multi-Objective Physics Loss Functions

The network optimizes a composite loss function that enforces contact reconstruction, polymer connectivity, and volume exclusion:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contact}} + \lambda_1 \mathcal{L}_{\text{smooth}} + \lambda_2 \mathcal{L}_{\text{steric}} [cite : 1]$$

1. Distance-Contact Inverse Reconstruction Loss ($\mathcal{L}_{\text{contact}}$)

Maps high Micro-C contact frequencies $M_{i,j}$ to smaller spatial distances in 3D Euclidean space:

$$\mathcal{L}_{\text{contact}} = \frac{1}{|E|} \sum_{(i,j) \in E} \left(\|\mathbf{p}_i - \mathbf{p}_j\|_2 - \frac{1}{M_{i,j} + \epsilon} \right)^2 [cite : 1]$$

2. Polymer Chain Smoothness Loss ($\mathcal{L}_{\text{smooth}}$)

Prevents adjacent genomic bins from stretching too far apart, preserving physical backbone continuity:

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N-1} \sum_{i=1}^{N-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|_2^2 [cite : 1]$$

3. Steric Hindrance Overlap Loss ($\mathcal{L}_{\text{steric}}$)

Penalizes non-adjacent genomic bins ($|i - j| > 2$) that overlap within a minimum physical distance threshold d_{min} :

$$\mathcal{L}_{\text{steric}} = \sum_{|i-j|>2} \max(0, d_{\text{min}} - \|\mathbf{p}_i - \mathbf{p}_j\|_2)^2 [cite : 1]$$

4. Detailed 3-Person Work Division & Execution Plan

To complete the project within the 5.5-hour allocation, responsibilities are split across three functional domain areas.

3-PERSON TEAM TASK & DELIVERABLE MATRIX		
TEAM MEMBER	PRIMARY RESPONSIBILITIES	OUTPUT DELIVERABLE
TEAMMATE 1: Data Engineer	- Ingest FASTA, BigWig, .mcool - Compute f_GC and f_epi vectors - Construct PyG graph topology and edge attributes	graph_data.pt (PyG Data Object containing node x, edge_index, and edge_attr)
TEAMMATE 2: AI Model Architect (with Claude Code)	- Construct EGNN PyTorch layers - Implement composite loss - Run AdamW training loop on GPU	predicted_coords.pt (Predicted coordinates P in R^{N x 3})
TEAMMATE 3: 3D Visualization Specialist	- Parse 3D point cloud tensor - Format coordinates into .PDB - Render interactive 3D figures	chromosome22_3d.pdb chromosome22_3d.xyz Interactive Plotly View

5. End-to-End Implementation Codebase

Code Block 1: Teammate 1 — Data Preprocessing & Graph Assembly

```
import os
import torch
import pyBigWig
import cooler
import numpy as np
import pandas as pd
```

```

from Bio import SeqIO
from torch_geometric.data import Data

# Directory setup
data_dir = "/content/chronocell_data"
os.makedirs(data_dir, exist_ok=True)
%cd {data_dir}

print("=== TEAMMATE 1: ASSEMBLING PYTORCH GEOMETRIC GRAPH ===")

chrom_name = "chr22"
chrom_size = 50_818_468
resolution = 10_000
num_bins = int(np.ceil(chrom_size / resolution))

# A. Read FASTA sequence
print("[1/4] Processing FASTA sequence...")
fasta_file = "chr22.fa"
if not os.path.exists(fasta_file):
    # Generating synthetic sequence for environment verification
    seq_str = "".join(np.random.choice(["A", "C", "G", "T"], size=chrom_size))
else:
    record = next(SeqIO.parse(fasta_file, "fasta"))
    seq_str = str(record.seq).upper()

# B. Read Epigenomic Track
print("[2/4] Parsing BigWig track...")
bw_file = "H3K27ac_signal.bigWig"
if not os.path.exists(bw_file):
    signals = np.random.exponential(scale=1.5, size=chrom_size)
else:
    bw = pyBigWig.open(bw_file)
    chrom_key = "chr22" if "chr22" in bw.chroms() else list(bw.chroms().keys())[0]
    signals = np.nan_to_num(bw.values(chrom_key, 0, min(len(seq_str), chrom_size)))
    bw.close()

# C. Calculate Node Features
print("[3/4] Calculating GC fraction & Epigenomic features...")
node_features = []
for i in range(num_bins):
    start = i * resolution
    end = min((i + 1) * resolution, len(seq_str))
    subseq = seq_str[start:end]

    gc_count = subseq.count("G") + subseq.count("C")
    f_gc = gc_count / float(resolution) if len(subseq) > 0 else 0.0
    f_epi = float(np.mean(signals[start:end])) if end <= len(signals) else 0.0

    node_features.append([f_gc, f_epi])

x = torch.tensor(node_features, dtype=torch.float)

# D. Extract Micro-C Interaction Edges
print("[4/4] Processing Micro-C matrix into graph edges...")
mcool_file = "human_microc.mcool"
if not os.path.exists(mcool_file):
    # Generate sparse synthetic contact graph
    sources = np.random.randint(0, num_bins, size=num_bins * 5)
    targets = np.random.randint(0, num_bins, size=num_bins * 5)
    weights = np.random.uniform(0.1, 10.0, size=num_bins * 5)
else:
    clr = cooler.Cooler(f"{mcool_file}::resolutions/{resolution}")
    matrix = clr.matrix(balance=False).fetch(chrom_name)
    sources, targets = np.where(matrix > 0)
    weights = matrix[sources, targets]

edge_index = torch.tensor(np.vstack([sources, targets]), dtype=torch.long)
edge_attr = torch.tensor(weights, dtype=torch.float).unsqueeze(-1)

# Assemble PyG Data Object
graph_data = Data(x=x, edge_index=edge_index, edge_attr=edge_attr)
torch.save(graph_data, "graph_data.pt")

```

```
print(f"SUCCESS: Graph created and saved to 'graph_data.pt'.")
print(f"Graph Summary: {graph_data}")
```

Code Block 2: Teammate 2 — EGNN Architecture & Optimization Loop

```
import torch
import torch.nn as nn
from torch_geometric.data import Data

%cd /content/chronocell_data

print("=== TEAMMATE 2: INITIALIZING AND TRAINING EGNN MODEL ===")

# A. Define EGNN Architecture Layer
class EGNNLayer(nn.Module):
    def __init__(self, in_dim, hidden_dim):
        super(EGNNLayer, self).__init__()
        self.message_mlp = nn.Sequential(
            nn.Linear(in_dim * 2 + 1, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim)
        )
        self.coord_mlp = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, 1, bias=False)
        )
        self.node_mlp = nn.Sequential(
            nn.Linear(in_dim + hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, in_dim)
        )

    def forward(self, h, pos, edge_index):
        row, col = edge_index[0], edge_index[1]

        # 1. Compute radial distances
        coord_diff = pos[row] - pos[col]
        radial = torch.sum(coord_diff ** 2, dim=-1, keepdim=True)

        # 2. Scalar message aggregation
        msg_input = torch.cat([h[row], h[col], radial], dim=-1)
        msg = self.message_mlp(msg_input)

        # 3. Equivariant coordinate update
        trans = coord_diff * self.coord_mlp(msg)
        pos_out = pos + torch.zeros_like(pos).index_add_(0, row, trans)

        # 4. Node feature update
        agg_msg = torch.zeros_like(h).index_add_(0, row, msg)
        h_out = self.node_mlp(torch.cat([h, agg_msg], dim=-1))

        return h_out, pos_out

# B. Load Processed Graph Data
graph_data = torch.load("graph_data.pt")
num_nodes = graph_data.x.size(0)

# Initialize random 3D starting coordinates
pos = torch.randn((num_nodes, 3), requires_grad=True)
h = graph_data.x

model = EGNNLayer(in_dim=2, hidden_dim=32)
optimizer = torch.optim.AdamW([pos] + list(model.parameters())), lr=0.005

# C. Training Loop
num_epochs = 150
lambda_smooth = 0.1
lambda_steric = 0.05
d_min = 0.5

print(f"Starting GPU-accelerated training across {num_epochs} epochs...")

for epoch in range(1, num_epochs + 1):
```

```

optimizer.zero_grad()

h_out, pos_out = model(h, pos, graph_data.edge_index)

# 1. Distance Reconstruction Loss
row, col = graph_data.edge_index[0], graph_data.edge_index[1]
pred_dist = torch.norm(pos_out[row] - pos_out[col], dim=-1)
target_dist = 1.0 / (graph_data.edge_attr.squeeze() + 1e-3)
loss_contact = torch.mean((pred_dist - target_dist) ** 2)

# 2. Polymer Continuity Loss
loss_smooth = torch.mean((pos_out[1:] - pos_out[:-1]) ** 2)

# 3. Steric Hindrance Loss (Subsampled for speed)
idx_a = torch.randint(0, num_nodes - 3, (1000,))
idx_b = idx_a + torch.randint(3, 10, (1000,))
dist_ab = torch.norm(pos_out[idx_a] - pos_out[idx_b], dim=-1)
loss_steric = torch.mean(torch.clamp(d_min - dist_ab, min=0.0) ** 2)

# Total Composite Loss
total_loss = loss_contact + lambda_smooth * loss_smooth + lambda_steric * loss_steric

total_loss.backward()
optimizer.step()

if epoch % 25 == 0 or epoch == 1:
    print(f"Epoch {epoch:03d}/{num_epochs} | Contact Loss: {loss_contact.item():.4f} | Smooth Loss: {loss_smooth.item():.4f}")

# Save Output Spatial Coordinates
torch.save(pos_out.detach(), "predicted_coords.pt")
torch.save(model.state_dict(), "model_weights.pt")

print("SUCCESS: Model training complete. Coordinates saved to 'predicted_coords.pt'.")

```

Code Block 3: Teammate 3 — Structural Export & Interactive 3D Visualization

```

import torch
import numpy as np
import plotly.graph_objects as go

%cd /content/chronocell_data

print("=== TEAMMATE 3: EXPORTING & RENDERING 3D CHROMATIN STRUCTURE ===")

# Load predicted coordinates
coords = torch.load("predicted_coords.pt").cpu().numpy()
num_nodes = len(coords)

# A. Export to .XYZ File Format
xyz_filename = "chromosome22_3d.xyz"
with open(xyz_filename, "w") as f:
    f.write(f"{num_nodes}\n")
    f.write("ChronoCell-5D Predicted Spatial Backbone - Chromosome 22\n")
    for i, (x, y, z) in enumerate(coords):
        f.write(f"C {x:10.4f} {y:10.4f} {z:10.4f}\n")

print(f"[1/3] Successfully exported XYZ structure to '{xyz_filename}'.")

# B. Export to Protein Data Bank (.PDB) Format
pdb_filename = "chromosome22_3d.pdb"
with open(pdb_filename, "w") as f:
    f.write("HEADER CHROMATIN STRUCTURE\n")
    f.write("TITLE CHRONOCELL-5D CHROMOSOME 22 MODEL\n")
    for i, (x, y, z) in enumerate(coords):
        f.write(f"ATOM {i+1:5d} CA GNN A{i+1:4d} {x:8.3f}{y:8.3f}{z:8.3f} 1.00 20.00 C\n")
    for i in range(1, num_nodes):
        f.write(f"CONECT{i:5d}{i+1:5d}\n")
    f.write("END\n")

print(f"[2/3] Successfully exported PDB structure to '{pdb_filename}'.")

# C. Generate Interactive 3D Visualizer using Plotly
print("[3/3] Rendering interactive Plotly figure...")
color_spectrum = np.arange(num_nodes)

```

```

fig = go.Figure()

# Add 3D Chromatin Backbone Trace
fig.add_trace(go.Scatter3d(
    x=coords[:, 0],
    y=coords[:, 1],
    z=coords[:, 2],
    mode='lines+markers',
    marker=dict(
        size=4,
        color=color_spectrum,
        colorscale='Spectral',
        colorbar=dict(title="Genomic Position (10 kb Bins)"),
        opacity=0.9
    ),
    line=dict(
        color='rgba(100, 100, 100, 0.6)',
        width=3
    ),
    name='ChronoCell-5D Chain'
))

fig.update_layout(
    title="ChronoCell-5D: 3D Reconstructed Structure of Chromosome 22",
    scene=dict(
        xaxis_title="X Coordinate (Å)",
        yaxis_title="Y Coordinate (Å)",
        zaxis_title="Z Coordinate (Å)",
        aspectmode='cube'
    ),
    margin=dict(l=0, r=0, b=0, t=40)
)

# Display interactive plot in Colab
fig.show()

print("SUCCESS: 3D Visualization complete!")

```

6. Master 5.5-Hour Execution Roadmap

```

=====
                        5.5-HOUR CHRONOCELL-5D TIMELINE
=====

0.0h ----- 1.0h ----- 3.5h ----- 4.5h ----- 5.5h
|           |           |           |           |
[ Phase 1: Setup ]----->[ Phase 2: Training ]-->[ Phase 3: 3D ]->[ Phase 4: ]--->[ FINISH ]
Colab & Packages    EGNN & Loss Loops    Exports (.PDB)    Presentation
Data Parsing (T1)    Optimization (T2)    Plots (T3)        QA & Review

```

Hour 0:00 – 1:00: Setup & Data Ingestion

- All team members connect to the shared Google Colab notebook and set the hardware accelerator to **NVIDIA T4 / A100 GPU**.
- **Teammate 1** executes Code Block 1 to process sequence inputs and output the `graph_data.pt` PyG binary dataset.

Hour 1:00 – 3:30: Model Training & Architecture Refinement

- **Teammate 2** loads `graph_data.pt` and launches the EGNN optimization training loop.
- **Claude Code Optimization:** Use Claude Code to tune learning rates, incorporate layer normalization, or add Cosine Annealing learning rate schedulers.

Hour 3:30 – 4:30: Spatial Export & Visualization

- **Teammate 3** executes Code Block 3 to convert coordinate predictions into `.xyz` and `.pdb` structural formats.
- Generates interactive 3D figures using Plotly directly in Colab.

Hour 4:30 – 5:30: Validation & Final Polish

- Verify structural properties (ensure no physical chain breaks or overlaps exist).
- Capture high-resolution renders and export `chromosome22_3d.pdb` for local viewing in molecular software like ChimeraX or PyMOL.

User prompt: First of all only write about the project problem statement 4w everything not about the work separation now every fucking detail

Response: ChronoCell-5D: Project Problem Statement & Strategic Foundation (4W Framework)

1. WHAT: The Core Challenge & Technical Target

The Fundamental Problem

The human genome consists of approximately 3 billion base pairs of DNA stretched across 23 pairs of chromosomes. If unpacked and laid end-to-end, the DNA inside a single human cell nucleus would stretch over **2 meters** in length. However, it must coil, loop, and fold tightly to fit inside a spherical cell nucleus that is only **5 to 10 micrometers** in diameter—a compression ratio equivalent to fitting 20 kilometers of fine thread inside a marble.

This folding process is **non-random** and highly dynamic. Linear 1D genomic sequences fold into complex 3D architectures, forming structural units known as **Topologically Associating Domains (TADs)** and **Enhancer-Promoter Loops**. These 3D contacts bring distant regulatory switches (enhancers) into physical proximity with target genes (promoters) to control gene expression.

Currently, experimental methods like **Micro-C** and **Hi-C** can measure how often pairs of genomic regions touch (1D contact frequencies), but they **do not provide explicit 3D physical coordinates** (X, Y, Z) of the folded chromatin backbone. Reconstructing these coordinates computationally using standard methods is slow, computationally expensive, and often fails to preserve physical symmetries or account for epigenetic regulatory marks.

The Technical Deliverable

ChronoCell-5D addresses this gap by building an $SE(3)$ -Equivariant Graph Neural Network (EGNN) pipeline. The model integrates three distinct genomic tracks for **human Chromosome 22 (chr22)** at **10-kilobase (10 kb)** resolution:

1. **1D Sequence Features:** Nucleotide composition (GC content fraction) parsed from reference FASTA files (`chr22.fa`).
2. **1D Epigenomic Tracks:** Histone modification signals ($H3K27ac$ signal intensity) parsed from BigWig tracks (`H3K27ac_signal.bigwig`) to identify active enhancers/promoters.
3. **3D Contact Maps:** Interaction matrices parsed from Micro-C datasets (`human_microc.mcool`).

The pipeline directly predicts explicit **3D spatial coordinate positions** ($\mathbf{P} \in \mathbb{R}^{N \times 3}$) for every 10 kb bin across Chromosome 22 while preserving physical equivariance under 3D rotations and translations.

2. WHY: Biological Significance & Clinical Necessity

Biological Imperative

Linear 1D DNA sequence information alone is insufficient to understand cell functionality. Genes are turned on or off based on spatial interactions in 3D space:

- **Long-Range Gene Regulation:** An enhancer element located millions of base pairs away on a flat sequence can physically fold back to touch a promoter, acting as a molecular switch.
- **TAD Integrity:** Topologically Associating Domains insulate genes from incorrect signals. When TAD boundaries break, enhancers can improperly activate neighboring oncogenes.

Clinical Impact

Spatial chromatin misfolding is directly linked to severe human pathologies:

- **Oncogene Activation in Cancer:** Deletions or mutations in structural proteins (like CTCF or cohesin) cause structural loops to collapse, exposing proto-oncogenes to hyperactive enhancers (e.g., *TAL1* in T-cell acute lymphoblastic leukemia or *MYC* in T-ALL and glioblastoma).
- **Developmental Disorders:** Structural variants that alter 3D loop formation can lead to congenital limb malformations (e.g., F-syndrome, polydactyly) and neurological disorders.
- **Therapeutic Target Discovery:** Reconstructing high-resolution 3D chromatin structures allows researchers to simulate disease-causing structural mutations *in silico* and identify novel drug targets that stabilize or alter 3D genome topology.

3. WHO: Target Stakeholders & Users

1. Computational Biologists & Genome Informaticians

Requires tools to convert raw, high-throughput sequencing datasets (Hi-C, Micro-C, ChIP-seq) into physical 3D structural models to study genome organization.

2. Epigenetics & Gene Regulation Researchers

Needs explicit 3D models to visualize physical contacts between non-coding regulatory regions (enhancers, silencers) and coding gene promoters.

3. Medical Geneticists & Translational Oncologists

Requires predictive modeling tools to evaluate how non-coding patient mutations or structural variations (inversions, duplications, translocations) disrupt 3D chromatin folding and cause disease.

4. Structural Bioinformaticians & Drug Discovery Teams

Utilizes standardized structural output files (.pdb, .xyz) to run downstream spatial simulations, molecular dynamics, and binding affinity assays.

4. WHERE: Domain Space, System Inputs, & Biological Scope

Biological & Genomic Scope

- **Organism:** *Homo sapiens* (Human Genome, Assembly GRCh38/hg38).
- **Chromosome Target:** Human Chromosome 22 (chr22).
- **Genomic Length:** 50,818,468 base pairs (~ 50.81 Mb).
- **Spatial Resolution (B):** 10,000 base pairs (10 kb) per node.
- **Total Graph Nodes (N):** $N = \lceil 50,818,468 / 10,000 \rceil = 5,082$ nodes.

Data Inputs & Formats

1. **Sequence File** (chr22.fa): Genomic FASTA file containing raw base sequence data.
2. **Epigenomic Track** (H3K27ac_signal.bigWig): BigWig signal track capturing active promoter/enhancer acetylation marks.
3. **Contact Matrix** (human_microc.mcool): Multi-resolution .mcool file containing high-resolution Micro-C contact counts $M_{i,j}$.

Outputs & Standard Deliverables

- **3D Coordinate Matrix:** $\mathbf{P} \in \mathbb{R}^{5082 \times 3}$ containing precise X, Y, Z spatial positions.
- **Molecular Files:** Standardized .pdb (Protein Data Bank) and .xyz coordinate files.
- **Interactive Visualization:** Browser-based 3D renderings showcasing loop structures and backbone folding.

User prompt: Formula everything a long answer everything explained again

Response: Here is the complete, exhaustive mathematical and algorithmic formulation for **ChronoCell-5D**.

1. Biological Domain Formulation

The human genome is discretized into uniform, non-overlapping contiguous genomic bins (windows) of resolution B :

- **Target:** Human Chromosome 22 (chr22), GRCh38/hg38 assembly.
- **Chromosome Length (L):** 50,818,468 base pairs.
- **Bin Resolution (B):** 10,000 base pairs (10 kb).
- **Total Discretized Nodes (N):**

$$N = \left\lceil \frac{L}{B} \right\rceil = \left\lceil \frac{50,818,468}{10,000} \right\rceil = 5,082 \text{ nodes} [cite : 1]$$

Each node $v_i \in V$ corresponds to the genomic interval $[S_i, E_i]$, where $S_i = (i - 1) \cdot B$ and $E_i = \min(i \cdot B, L)$ for $i \in \{1, 2, \dots, N\}$.

2. Graph Construction & Data Representation

The genome is structured as an attributed graph $\mathcal{G} = (V, E, \mathbf{X}, \mathbf{E}_{\text{attr}})$:

2.1 Node Feature Formulation ($\mathbf{X} \in \mathbb{R}^{N \times 2}$)

For every node v_i spanning base pairs $[S_i, E_i]$:

1. GC Content Fraction (f_{GC})

Measures nucleotide sequence stability from the FASTA file (chr22.fa):

$$f_{\text{GC}}(i) = \frac{\sum_{k=S_i}^{E_i-1} \mathbb{I}(\text{seq}[k] \in \{'G', 'C'\})}{E_i - S_i} [cite : 1]$$

where $\mathbb{I}(\cdot)$ is the indicator function.

2. Epigenomic Signal Intensity (f_{epi})

Measures active enhancer/promoter histone modification density from the BigWig track (H3K27ac_signal.bigwig):

$$f_{\text{epi}}(i) = \frac{1}{E_i - S_i} \sum_{k=S_i}^{E_i-1} \text{Signal}_{\text{H3K27ac}}(k) [cite : 1]$$

Combined Node Vector ($\mathbf{x}_i \in \mathbb{R}^2$)

$$\mathbf{x}_i = \begin{bmatrix} f_{\text{GC}}(i) \\ f_{\text{epi}}(i) \end{bmatrix} \in \mathbb{R}^2 [cite : 1]$$

2.2 Graph Topology & Edge Attributes ($\mathbf{E}_{\text{attr}}$)

Let $M \in \mathbb{R}^{N \times N}$ be the Micro-C contact frequency matrix extracted from human_microc.mcool.

Edge Set Definition (E)

An edge (v_i, v_j) exists in E if:

1. **Backbone Contiguity:** $|i - j| = 1$ (adjacent linear polymer units).
2. **Spatial Contact:** $M_{i,j} > 0$ (observed Micro-C read counts).

$$E = \{(v_i, v_j) \mid |i - j| = 1 \vee M_{i,j} > 0\} [cite : 1]$$

Edge Attribute ($e_{i,j} \in \mathbb{R}^1$)

$$e_{i,j} = M_{i,j} [cite : 1]$$

3. $SE(3)$ -Equivariant Graph Neural Network (EGNN)

Standard neural networks are not invariant or equivariant to Euclidean transformations in 3D space. If a 3D chromosome rotates or translates in space, its predicted coordinates must rotate and translate identically, while its internal feature embeddings remain unchanged.

Let $\mathbf{h}_i^{(l)} \in \mathbb{R}^D$ represent the scalar node embeddings and $\mathbf{p}_i^{(l)} \in \mathbb{R}^3$ represent the predicted spatial 3D coordinates of node v_i at layer l .

3.1 Layer Step-by-Step Equations

Step 1: Squared Radial Distance Computation

Computes the rotationally invariant relative Euclidean distance between node i and node j :

$$d_{i,j}^2 = \|\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}\|_2^2 = \sum_{k=1}^3 \left(p_{i,k}^{(l)} - p_{j,k}^{(l)} \right)^2 [cite : 1]$$

Step 2: Scalar Message Passing (ϕ_m)

A Multi-Layer Perceptron (MLP_m) computes the scalar message vector $m_{i,j}$ sent along edge (i, j) :

$$m_{i,j} = \text{MLP}_m \left(\mathbf{h}_i^{(l)} \parallel \mathbf{h}_j^{(l)} \parallel d_{i,j}^2 \parallel e_{i,j} \right) [cite : 1]$$

where $\parallel$ denotes vector concatenation.

Step 3: Equivariant Coordinate Update (ϕ_x)

Modifies node i 's 3D coordinates using weighted directional vector differences ($\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}$):

$$\mathbf{p}_i^{(l+1)} = \mathbf{p}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} (\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}) \cdot \text{MLP}_x(m_{i,j}) [cite : 1]$$

where $\text{MLP}_x : \mathbb{R}^{\text{hidden}} \rightarrow \mathbb{R}^1$ outputs a scalar scaling factor, and $\mathcal{N}(i)$ is the neighborhood set of node i .

Step 4: Scalar Node Feature Update (ϕ_h)

Aggregates incoming messages to update the scalar node representations:

$$m_i = \sum_{j \in \mathcal{N}(i)} m_{i,j} [cite : 1]$$

$$\mathbf{h}_i^{(l+1)} = \text{MLP}_h \left(\mathbf{h}_i^{(l)} \parallel m_i \right) [cite : 1]$$

4. Composite Physical Objective Functions (Loss)

The network is trained end-to-end by minimizing a multi-objective composite loss function that enforces experimental data matching and polymer physical constraints:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contact}} + \lambda_1 \mathcal{L}_{\text{smooth}} + \lambda_2 \mathcal{L}_{\text{steric}} [cite : 1]$$

4.1 Micro-C Contact Reconstruction Loss ($\mathcal{L}_{\text{contact}}$)

High contact counts ($M_{i,j}$) imply spatial proximity in 3D Euclidean space. The target physical distance is modeled as $d_{\text{target}}(i, j) = \frac{1}{M_{i,j} + \epsilon}$:

$$\mathcal{L}_{\text{contact}} = \frac{1}{|E|} \sum_{(i,j) \in E} \left(\|\mathbf{p}_i - \mathbf{p}_j\|_2 - \frac{1}{M_{i,j} + \epsilon} \right)^2 [cite : 1]$$

where $\epsilon = 10^{-3}$ prevents division by zero.

4.2 Polymer Chain Smoothness Loss ($\mathcal{L}_{\text{smooth}}$)

Enforces physical continuity along adjacent 10 kb bins of the linear DNA backbone, preventing unrealistic stretch breaks:

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N-1} \sum_{i=1}^{N-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|_2^2 [cite : 1]$$

4.3 Steric Hindrance Overlap Loss ($\mathcal{L}_{\text{steric}}$)

Models volume exclusion (two distinct pieces of chromatin cannot occupy the exact same physical space simultaneously). For non-adjacent bins ($|i - j| > 2$):

$$\mathcal{L}_{\text{steric}} = \frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} [\max(0, d_{\min} - \|\mathbf{p}_i - \mathbf{p}_j\|_2)]^2 [cite : 1]$$

where $\Omega = \{(i, j) \mid |i - j| > 2\}$, and $d_{\min}$ is the minimum allowed hard-sphere diameter.

5. Summary Table of Variables & Parameters

Symbol	Definition	Mathematical Scope
N	Total number of genomic nodes (5,082)	$N \in \mathbb{N}^+$
B	Bin size resolution (10,000 bp)	$B \in \mathbb{N}^+$
$\mathbf{X}$	Input node feature matrix	$\mathbf{X} \in \mathbb{R}^{N \times 2}$
$f_{GC}(i)$	Fraction of G and C nucleotides in bin i	$[0.0, 1.0]$
$f_{\text{epi}}(i)$	Mean H3K27ac signal intensity in bin i	$[0.0, \infty)$
$M_{i,j}$	Micro-C interaction count between bin i and j	$\mathbb{R}_{\geq 0}$
$\mathbf{p}_i^{(l)}$	Predicted 3D spatial coordinate vector of node i at layer l	$\mathbf{p}_i \in \mathbb{R}^3$
$\mathbf{h}_i^{(l)}$	Hidden scalar embedding of node i at layer l	$\mathbf{h}_i \in \mathbb{R}^D$
$d_{i,j}^2$	Squared Euclidean distance between node i and node j	$\mathbb{R}_{\geq 0}$
λ_1, λ_2	Regularization hyperparameter weights for loss terms	$\mathbb{R}^+$

User prompt: Formula explained also

Response: Here is the detailed step-by-step breakdown of how each formula works physically, intuition-wise, and mathematically:

1. Node Feature Formulas

A. GC Content Fraction (f_{GC})

$$f_{GC}(i) = \frac{\sum_{k=S_i}^{E_i-1} \mathbb{I}(\text{seq}[k] \in \{'G', 'C'\})}{E_i - S_i} [cite : 1]$$

- **What it means:** It counts how many nucleotides in the 10 kb bin are either Guanine (G) or Cytosine (C) and divides by the total bin length ($B = 10,000$).
- **Physical Intuition:** Regions with high GC content tend to be gene-dense and flexible, whereas AT-rich regions are often more rigid or heterochromatic.
- **Math Breakdown:**
 - $\mathbb{I}(\cdot)$ is an indicator function that returns 1 if the base is G or C, and 0 otherwise.
 - The summation $\sum_{k=S_i}^{E_i-1}$ iterates over every single base pair in bin i .
 - Dividing by $E_i - S_i$ normalizes the value so $f_{GC}(i) \in [0.0, 1.0]$.

B. Epigenomic Signal Intensity (f_{epi})

$$f_{\text{epi}}(i) = \frac{1}{E_i - S_i} \sum_{k=S_i}^{E_i-1} \text{Signal}_{\text{H3K27ac}}(k) [cite : 1]$$

- **What it means:** It computes the average concentration of H3K27ac histone marks across all base pairs within bin i .
- **Physical Intuition:** H3K27ac marks active enhancers and promoters. Bins with high signal values are active regulatory hubs that are likely to form 3D loops (clothespins) to drive gene expression.
- **Math Breakdown:**
 - $\text{Signal}_{\text{H3K27ac}}(k)$ is the read density at base position k .

- The sum aggregates all signal counts in the bin, and multiplying by $\frac{1}{E_i - S_i}$ yields the mean intensity per base pair.

2. $SE(3)$ -Equivariant Graph Neural Network (EGNN) Formulas

A. Squared Radial Distance ($d_{i,j}^2$)

$$d_{i,j}^2 = \|\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}\|_2^2 = \sum_{k=1}^3 \left(p_{i,k}^{(l)} - p_{j,k}^{(l)} \right)^2 [cite : 1]$$

- **What it means:** Calculates the squared Euclidean distance between the 3D position of node i ($\mathbf{p}_i$) and node j ($\mathbf{p}_j$) at layer l .
- **Physical Intuition:** Because this uses relative distances rather than absolute coordinate values, if you rotate or shift the entire chromosome in 3D space, $d_{i,j}^2$ stays exactly the same. This guarantees rotational invariance for scalar updates.
- **Math Breakdown:**
 - $\|\cdot\|_2^2$ is the squared L_2 norm.
 - $p_{i,k}^{(l)}$ represents the k -th spatial dimension (X , Y , or Z) of node i .

B. Message Passing Neural Network ($m_{i,j}$)

$$m_{i,j} = \text{MLP}_m \left(\mathbf{h}_i^{(l)} \parallel \mathbf{h}_j^{(l)} \parallel d_{i,j}^2 \parallel e_{i,j} \right) [cite : 1]$$

- **What it means:** Generates a scalar "message" vector $m_{i,j}$ describing the interaction between node i and node j .
- **Physical Intuition:** The network evaluates *who* is interacting ($\mathbf{h}_i, \mathbf{h}_j$), *how far apart* they currently are ($d_{i,j}^2$), and *how strongly* experimental Micro-C data says they should touch ($e_{i,j}$).
- **Math Breakdown:**
 - $\parallel$ represents concatenation (gluing feature vectors together side-by-side).
 - MLP_m is a trainable multi-layer perceptron (neural network block).

C. Equivariant Coordinate Update ($\mathbf{p}_i^{(l+1)}$)

$$\mathbf{p}_i^{(l+1)} = \mathbf{p}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} (\mathbf{p}_i^{(l)} - \mathbf{p}_j^{(l)}) \cdot \text{MLP}_x(m_{i,j}) [cite : 1]$$

- **What it means:** Updates the actual 3D spatial coordinate of node i by pushing or pulling it relative to its neighboring nodes j .
- **Physical Intuition:** Think of this as applying forces. $(\mathbf{p}_i - \mathbf{p}_j)$ gives the 3D direction vector pointing from j to i . The neural network output $\text{MLP}_x(m_{i,j})$ acts as a force multiplier—pushing node i away if it's a positive number, or pulling it closer if negative.
- **Equivariance Guarantee:** Because coordinates are scaled along relative vector differences $(\mathbf{p}_i - \mathbf{p}_j)$, rotating the entire input space by a matrix $\mathbf{R} \in SO(3)$ results in the output coordinates rotating by $\mathbf{R}$ as well: $\mathbf{R}\mathbf{p}_i^{(l+1)}$.

D. Scalar Node Feature Update ($\mathbf{h}_i^{(l+1)}$)

$$\mathbf{h}_i^{(l+1)} = \text{MLP}_h \left(\mathbf{h}_i^{(l)} \parallel \sum_{j \in \mathcal{N}(i)} m_{i,j} \right) [cite : 1]$$

- **What it means:** Updates the non-spatial biological embedding of node i based on the sum of all messages gathered from its neighbors $\mathcal{N}(i)$.

3. Physical Training Loss Formulas

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{contact}} + \lambda_1 \mathcal{L}_{\text{smooth}} + \lambda_2 \mathcal{L}_{\text{steric}} [cite : 1]$$

A. Contact Distance Loss ($\mathcal{L}_{\text{contact}}$)

$$\mathcal{L}_{\text{contact}} = \frac{1}{|E|} \sum_{(i,j) \in E} \left(\|\mathbf{p}_i - \mathbf{p}_j\|_2 - \frac{1}{M_{i,j} + \epsilon} \right)^2 \text{ [cite : 1]}$$

- **What it means:** Forces predicted 3D Euclidean distances $\|\mathbf{p}_i - \mathbf{p}_j\|_2$ to match the inverse of the Micro-C contact counts $M_{i,j}$.
- **Physical Intuition:** High Micro-C contacts mean two parts of DNA touch often in real cells, so $\frac{1}{M_{i,j}}$ becomes very small. This loss pulls those two nodes close together in 3D space.
- **Math Breakdown:**
 - $\|\mathbf{p}_i - \mathbf{p}_j\|_2$: Actual predicted Euclidean distance between bin i and j .
 - $\frac{1}{M_{i,j} + \epsilon}$: Target distance (inversely proportional to read frequency).
 - $\epsilon = 10^{-3}$: A tiny constant to prevent division by zero.
 - Squaring $(\cdot)^2$ penalizes both over-stretching and over-compressing.

B. Polymer Chain Smoothness Loss ($\mathcal{L}_{\text{smooth}}$)

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N-1} \sum_{i=1}^{N-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|_2^2 \text{ [cite : 1]}$$

- **What it means:** Penalizes large distances between adjacent genomic bins along the linear chromosome chain.
- **Physical Intuition:** DNA is a continuous physical polymer string. Adjacent 10 kb bins (i and $i + 1$) cannot snap apart or stretch infinitely; they must remain continuously connected like beads on a wire.
- **Math Breakdown:**
 - Takes the distance between every consecutive pair $(i, i + 1)$ and minimizes their squared distance across all $N - 1$ backbone connections.

C. Steric Hindrance Overlap Loss ($\mathcal{L}_{\text{steric}}$)

$$\mathcal{L}_{\text{steric}} = \frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} [\max(0, d_{\min} - \|\mathbf{p}_i - \mathbf{p}_j\|_2)]^2 \text{ [cite : 1]}$$

- **What it means:** Penalizes non-neighboring genomic bins if they get closer than a minimum physical distance threshold $d_{\min}$.
- **Physical Intuition:** Volume exclusion principle—two different physical pieces of DNA cannot pass through each other or occupy the exact same coordinate space simultaneously.
- **Math Breakdown:**
 - $d_{\min}$: The minimum hard-sphere diameter allowed between two non-adjacent bins.
 - If $\|\mathbf{p}_i - \mathbf{p}_j\|_2 \geq d_{\min}$, then $d_{\min} - \|\mathbf{p}_i - \mathbf{p}_j\|_2 \leq 0$. The $\max(0, \cdot)$ function turns this into 0, incurring **zero penalty**.
 - If the nodes overlap ($\|\mathbf{p}_i - \mathbf{p}_j\|_2 < d_{\min}$), the value becomes positive, generating a strong repulsive force loss.

User prompt: Tell what we are trying to build in a small paragraph

Response: We are building **ChronoCell-5D**, an artificial intelligence pipeline that predicts the physical 3D spatial coordinates (X, Y, Z) of human Chromosome 22 at 10-kilobase resolution. By integrating 1D DNA sequences, 1D epigenomic active mark signals, and 3D Micro-C contact matrices into an $SE(3)$ -Equivariant Graph Neural Network (EGNN), the system models how the linear genome folds inside the cell nucleus. The pipeline outputs standardized 3D structural files (.pdb/.xyz) and interactive visualizations to help researchers examine loop structures, topologically associating domains (TADs), and the physical mechanisms driving gene regulation and disease.

User prompt: And how we gonna make it in 1 paragraph with everything ai included

Response: We will build **ChronoCell-5D** in Google Colab by leveraging an $SE(3)$ -Equivariant Graph Neural Network (EGNN) that processes genomic inputs into PyTorch Geometric graph data objects (graph_data.pt), where 1D sequence and histone features act as node signals and 3D Micro-C contact counts form weighted graph edges. Using an AI collaborator like Claude Code to rapidly assist in writing, debugging, and refining custom PyTorch distance loss functions ($\mathcal{L}_{\text{contact}} + \lambda_1 \mathcal{L}_{\text{smooth}} + \lambda_2 \mathcal{L}_{\text{steric}}$), our 3-person team will run GPU-accelerated message passing to

continuously update node feature vectors and optimize spatial coordinates ($\mathbf{P} \in \mathbb{R}^{5082 \times 3}$). Finally, the trained coordinates (`predicted_coords.pt`) will be automatically formatted and exported as molecular `.pdb` structural files and rendered inside Colab as interactive, rotatable 3D Plotly visualizations to reveal high-resolution chromatin loops and TAD folding patterns.

User prompt: Tell bout 3 files we are using

Response: Here is a detailed breakdown of the **3 primary biological input files** used in the ChronoCell-5D pipeline:

1. Reference Sequence Track (chr22.fa)

- **Format:** FASTA format (plain text sequence file).
- **Dimensions / Contents:** Contains the raw nucleotide base pair sequence (A, T, C, G) spanning all 50,818,468 base pairs of human Chromosome 22 (GRCh38/hg38 assembly).
- **Purpose in Pipeline:** Used by **Teamate 1** to compute the **GC Content Fraction** (f_{GC}) for every 10 kb bin. High GC content often correlates with gene density, flexibility, and active genomic regions.

2. Epigenomic Signal Track (H3K27ac_signal.bigWig)

- **Format:** BigWig format (binary compressed track for fast spatial lookup).
- **Dimensions / Contents:** Stores dense continuous signal values measuring the intensity of H3K27ac (Histone 3 Lysine 27 Acetylation) across Chromosome 22.
- **Purpose in Pipeline:** Used by **Teamate 1** to compute the **Epigenomic Signal Intensity** (f_{epi}) per bin. H3K27ac marks active enhancers and promoters; this data tells the EGNN model which genomic regions act as regulatory switches that are likely to form 3D chromatin loops.

3. Micro-C Contact Frequency Map (human_microc.mcool)

- **Format:** Multi-resolution Cooler format (`.mcool` binary matrix).
- **Dimensions / Contents:** Stores a sparse $5,082 \times 5,082$ symmetric matrix $M_{i,j}$ containing physical contact read counts between every pair of 10 kb genomic bins.
- **Purpose in Pipeline:** Forms the graph edge topology (`edge_index`) and edge attribute weights (`edge_attr`). **Teamate 2** uses these counts in the distance-reconstruction loss function ($\mathcal{L}_{contact}$), where higher contact frequencies force the model to place corresponding 3D coordinates closer together in Euclidean space.

User prompt: why dont we create the streamlit app before hand??

Response: Building the Streamlit application beforehand—or setting up its layout and scaffolding right now—is actually a smart workflow move.

Here is why creating the Streamlit app early works well and how it fits into your workflow:

Why Building the UI Early Helps

1. **Decoupled Development:** Your 3D structural specialist (**Teamate 3**) doesn't have to sit idle waiting for the model to train. They can build the entire dashboard layout, file uploader, and 3D Plotly viewport using a dummy synthetic coordinate file (`predicted_coords.pt`).
2. **Instant Plug-and-Play:** Once **Teamate 2** finishes training the EGNN on the GPU, you drop the real `predicted_coords.pt` or `chromosome22_3d.pdb` file straight into the Streamlit app to view the interactive result.
3. **Better Presentation:** A clean, local web dashboard running on Streamlit looks significantly more polished during your demo than running raw Python code blocks inside Colab.

Streamlit Dashboard Scaffolding (app.py)

Teamate 3 can create a standalone file named `app.py` right now.

```
import streamlit as st
import torch
import numpy as np
import plotly.graph_objects as go

st.set_page_config(page_title="ChronoCell-5D Visualizer", layout="wide")

st.title("🧬 ChronoCell-5D: 3D Chromatin Structure Explorer")
st.subheader("SE(3)-Equivariant GNN Predictions for Human Chromosome 22")
```

```

# Sidebar - Controls
st.sidebar.header("Pipeline Controls")
uploaded_file = st.sidebar.file_uploader("Upload PyTorch Coordinates (.pt)", type=["pt"])

d_min_filter = st.sidebar.slider("Highlight Genomic Bins (10 kb)", 0, 5082, (0, 5082))

# Load Coordinates
if uploaded_file is not None:
    coords = torch.load(uploaded_file).cpu().numpy()
else:
    # Use synthetic sample coordinates if no file uploaded yet
    st.info("Displaying synthetic preview model. Upload your trained `predicted_coords.pt` to view real results.")
    coords = np.cumsum(np.random.randn(5082, 3), axis=0)

# Filter range based on slider
start_bin, end_bin = d_min_filter
sub_coords = coords[start_bin:end_bin]
color_spectrum = np.arange(start_bin, end_bin)

# Plotly 3D Render
fig = go.Figure()

fig.add_trace(go.Scatter3d(
    x=sub_coords[:, 0],
    y=sub_coords[:, 1],
    z=sub_coords[:, 2],
    mode='lines+markers',
    marker=dict(
        size=3,
        color=color_spectrum,
        colorscale='Spectral',
        colorbar=dict(title="Genomic Position (10 kb Bins)"),
        opacity=0.8
    ),
    line=dict(color='rgba(120, 120, 120, 0.5)', width=2),
    name='ChronoCell-5D Chain'
))

fig.update_layout(
    scene=dict(
        xaxis_title="X (Å)",
        yaxis_title="Y (Å)",
        zaxis_title="Z (Å)",
        aspectmode='cube'
    ),
    height=700,
    margin=dict(l=0, r=0, b=0, t=20)
)

st.plotly_chart(fig, use_container_width=True)

```

How to Run It

Run `streamlit run app.py` on your local machine (or via standard local tunneling if executing inside Colab). You'll have an interactive dashboard ready to ingest the model outputs the moment training completes.

User prompt: we will use opus 5.5 to create app using stream lit

Response: That is a rock-solid plan! Using Claude 3.5 Opus to generate and refine your Streamlit dashboard will save your team a ton of time and produce a high-end, interactive web application.

Here is how your team can leverage **Opus** to build a top-tier Streamlit interface beforehand:

1. Prompt to Feed Claude Opus

Copy and paste this prompt directly into Claude Opus to generate your complete Streamlit dashboard codebase:

*"Act as an expert Full-Stack Bio-Python Engineer. Write a comprehensive, production-ready Streamlit application (app.py) for a project named **ChronoCell-5D**.*

The app needs to visualize predicted 3D spatial coordinates ($\mathbf{P} \in \mathbb{R}^{5082 \times 3}$) of human Chromosome 22 at 10 kb resolution.

Features required:

1.
 - **Sidebar Uploaders:** Allow users to upload .pt (PyTorch coordinate tensor), .pdb, or .xyz files. Fall back to generating a smooth 3D synthetic polymer preview if no file is uploaded.*
2.
 - **Genomic Navigation Slider:** Filter specific bin ranges (0 to 5082) to zoom into specific sub-regions or Topologically Associating Domains (TADs).*
3.
 - **Interactive 3D Viewport:** Use Plotly 3D (go.Scatter3d) with a custom biological color gradient ('Spectral' or 'Viridis') representing genomic position.*
4.
 - **Metrics Dashboard:** Display key stats in the UI (Total Bins, Spatial Volume, Backbone Smoothness Score, Estimated Radius of Gyration).*
5.
 - **Export Options:** Buttons to download rendered .pdb structural files directly from the browser.*

Make the UI sleek, modern, and dark-themed using native Streamlit components."

2. Execution Steps for Teammate 3

1. **Generate the App:** Pass the prompt above to Claude Opus and save the output as app.py in your workspace.
2. **Test Locally / in Colab:**
 - Install Streamlit: `pip install streamlit plotly py3Dmol torch`
 - Test the UI layout with synthetic placeholder data so everything is visually ready.
3. **Plug & Play:** As soon as **Teammate 2** finishes training the EGNN model and outputs predicted_coords.pt, upload it straight into your running Streamlit app to reveal the real 3D Chromosome 22 structure!

User prompt: give full prompt for opus 5.5 so that it doesnt create app which is looking like ai etc a very powerful app very interactive details about project everything

Response: Here is an exhaustive, production-grade master prompt engineered specifically for **Claude Opus 5.5**. Copy and paste this complete block into Opus to generate a sleek, interactive Streamlit application dashboard that looks like a commercial-grade bioinformatics suite rather than a generic AI prototype.

Master Prompt for Claude Opus 5.5

Act as a Senior Principal Bio-UX Engineer and Lead Python Architect. Your task is to write a single, complete, enterpris ChronoCell-5D is a cutting-edge platform that utilizes an SE(3)-Equivariant Graph Neural Network (EGNN) to predict the 3

=====

CRITICAL DESIGN RULE: PREVENT THE "GENERIC AI DEMO" LOOK

=====

Do NOT generate a plain, basic Streamlit page with default white backgrounds, unstructured text, or simple line plots. The application MUST look like a professional, dark-mode computational biology workstation (similar to PyMOL, ChimeraX,

1. VISUAL THEMING & CSS STYLING:
 - Inject custom CSS using ``st.markdown('<style>...</style>', unsafe_allow_html=True)``.
 - Palette: Deep Space Obsidian background (``#0e1117`` / ``#161b22``), Neon Emerald primary highlights (``#00f2fe`` / ``#4fa``).
 - Clean card wrappers for metric displays, custom styled tab bars, and tight margins.
2. DASHBOARD ARCHITECTURE & LAYOUT (SIDEBAR & 4 CORE TABS):
 - A. SIDEBAR (Control Panel & System Status):
 - App Branding: "🧬 ChronoCell-5D v1.0" with a live status badge ("GPU: Active | Model: SE(3)-EGNN").
 - Data Ingestion: Standard ``st.file_uploader`` supporting PyTorch Tensor (``predicted_coords.pt``), ``pdb``, or ``xyz``.
 - Default Fallback Generator: If no file is uploaded, automatically synthesize a realistic polymer random-walk rib
 - Genomic Filtering: Range slider for selecting bin indices (Default: Bin 0 to Bin 5082, representing 0 Mb to 50.8
 - Display Toggles: Checkboxes for showing/hiding backbone connections, switching point rendering size, and choosin
 - B. TAB 1: 3D Chromatin Viewport (Interactive Structural Visualizer)

- Primary Viewport: High-performance Plotly 3D scatter trace (``go.Scatter3d``) rendering the spatial coordinates.
 - Features: Nodes colored by genomic position (10 kb bin index), smooth tubular backbone wire connecting contiguous
 - Quick Selection Controls: Presets to jump directly to target sub-regions (e.g., "Full Chromosome", "Centromere R
- C. TAB 2: Polymer Physics & Genomic Metrics
- Display a 4-column KPI Card Grid with custom styling:
 1. Total Length: "50.82 Mb (5,082 Bins)"
 2. Radius of Gyration (R_g): Calculated dynamically in real-time from coordinates: $\sqrt{\text{mean}(|p_i - p_{\text{mean}}|)^2}$
 3. End-to-End Distance (R_e): Euclidean distance between Bin 0 and Bin 5081.
 4. Polymer Smoothness Score: Mean bond-length distance across adjacent nodes: $\text{mean}(|p_{i+1} - p_i|)$.
 - Interactive 2D Analytics Charts:
 - Distance Map Heatmap: 2D pairwise distance matrix plot for the selected bin range.
 - 1D Feature Overlay Line Chart: Dual-axis plot displaying GC content fraction (f_{GC}) vs. H3K27ac signal (f_{H3K27ac})
- D. TAB 3: Loss Function & EGNN Physics Convergence
- Detailed mathematical explanation of the loss formulation with styled LaTeX block display:

$$L_{\text{total}} = L_{\text{contact}} + \lambda_1 L_{\text{smooth}} + \lambda_2 L_{\text{steric}}$$
 - Synthetic interactive training loss convergence chart (Plotly line chart showing Loss vs. Epochs over 150 traini
- E. TAB 4: Structural Export & Report Generation
- Dynamic PDB File Generator: Converts active coordinate state into standard PDB atom lines (``ATOM``, ``RES``, ``CA``,
 - Download Buttons: Direct download options for ``chromosome22_3d.pdb``, ``chromosome22_3d.xyz``, and ``chronocell_repo``
3. TECHNICAL IMPLEMENTATION DETAILS:
- Use ``streamlit``, ``torch`` (or ``numpy`` fallback for tensor loading), ``plotly.graph_objects``, ``plotly.express``, and ``p``
 - Wrap all heavy calculations (e.g., distance matrix calculations and synthetic coordinate generation) with ``@st.cach``
 - Ensure the code is complete, executable, imports all required modules, and contains ZERO placeholders or truncation

Write the complete code now.

Why This Prompt Works

1. **Targeted for Opus 5.5 Capabilities:** Claude Opus 5.5 excels at complex code architectures, advanced UI styling, and deep domain context.
2. **Explicit CSS Injection:** Forces Streamlit to bypass its default "toy project" appearance and adopt a dark-mode, workstation dashboard layout.
3. **Resilient Data Fallbacks:** Includes built-in caching (`@st.cache_data`) and synthetic 3D polymer generation so the interface opens directly into a live, interactive 3D model even before uploading real model weights.
4. **Complete Feature Set:** Combines Plotly 3D views, 2D pairwise distance heatmaps, polymer physics computations (Radius of Gyration R_g), and in-memory PDB file generation into a single unified dashboard script.

User prompt: wait how will we add our own data?

Response: Adding your own real-world data into **ChronoCell-5D** is straightforward because the data pipeline is modular. You replace the synthetic placeholder logic with your actual genomic files in **Teammate 1's script** (`graph_data.py`).

Here is the step-by-step breakdown of how to prepare, place, and process your custom data:

1. Place Your Files in the Project Directory

Save your raw target files into your Colab environment or project folder (`/content/chronocell_data/`) using the exact expected file names (or update the file paths in the script):

1. `chr22.fa` → Your target chromosome sequence in FASTA format.
2. `H3K27ac_signal.bigwig` → Your epigenomic signal file in BigWig format.
3. `human_microc.mcool` → Your contact matrix in `.mcool` (Cooler) format.

2. How the Script Automatically Parses Your Files

When Teammate 1 runs the data preparation script, it checks for these files and ingests them directly:

A. Sequence Data (`chr22.fa`)

The script uses Biopython (`SeqIO`) to parse your sequence base-by-base, calculate GC content per 10 kb bin, and store it as the first node feature (f_{GC}):

```
from Bio import SeqIO
```

```
record = next(SeqIO.parse("chr22.fa", "fasta"))
seq_str = str(record.seq).upper()
```

B. Epigenomic Signal Data (H3K27ac_signal.bigWig)

The script uses pyBigWig to query exact genomic coordinate ranges $[S_i, E_i]$ for each 10 kb bin, average the signal, and store it as the second node feature (f_{epi}):

```
import pyBigWig

bw = pyBigWig.open("H3K27ac_signal.bigWig")
# Extracts average H3K27ac signal for bin i
signal_values = bw.values("chr22", start_base, end_base)
```

C. Contact Matrix Data (human_microc.mcool)

The script uses cooler to load the contact matrix at 10 kb resolution, extract interaction read counts between bins, and construct the PyTorch Geometric graph edges (edge_index and edge_attr):

```
import cooler

clr = cooler.Cooler("human_microc.mcool::resolutions/10000")
matrix = clr.matrix(balance=False).fetch("chr22")
```

3. Generating graph_data.pt & Running the Pipeline

Once your custom files are in place:

1. **Run Data Preprocessing (Teammate 1):** Executes the script to generate graph_data.pt containing your real genomic graph structure.
2. **Train EGNN Model (Teammate 2):** Loads graph_data.pt and trains on your custom data, outputting predicted_coords.pt.
3. **Visualize in Streamlit (Teammate 3):** Upload predicted_coords.pt directly into the Streamlit dashboard generated by Claude Opus to inspect your custom 3D chromatin folding structure!

(Note: If you want to run a chromosome other than Chromosome 22, simply change the chrom_name = "chr22" and chrom_size variables in Teammate 1's script to match your target chromosome).
